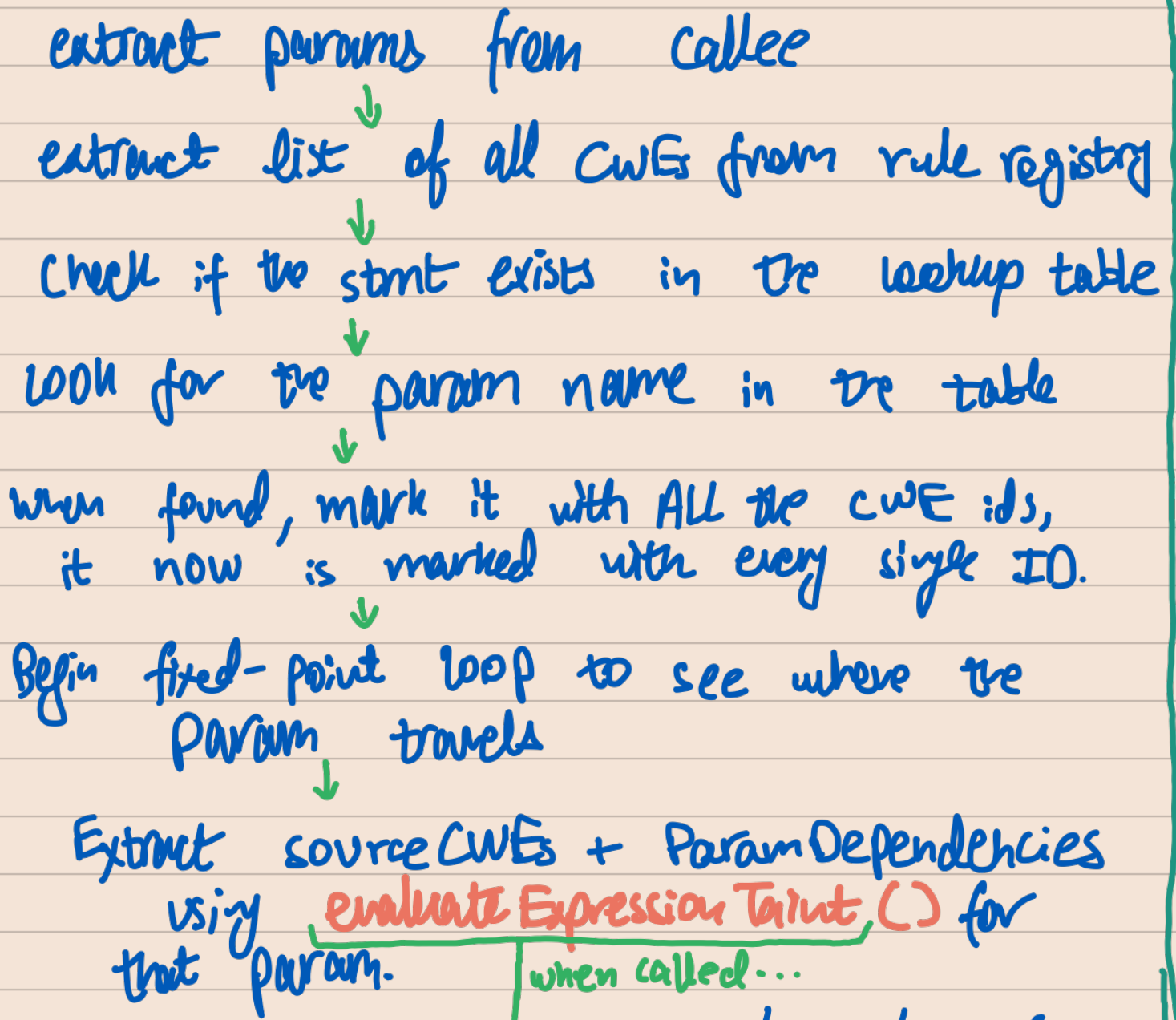


Here is the definitive flow used by vulnAI. I used this chart for whenever I got lost (which was alot).

I hope you like it :). It took me a week of pulling 6-hour days to fully understand, map out, design, and implement this.

Yet, this was incredibly fun and fulfilling.
Yes, my brain hurts.

Function Summary Builder



ast-call

vars / const / others

Extract the str name
using `getCallName()`

Extract args

we will eventually hit a var
being read

look at var-def edges

it returns Def of var

check where the Def of this
is in the tainted map
we stored it in

The srcCues and paramDeps
set filled with ALL the cues
we initially added.

Move up the call stack

children's cues get added to parent's maps

Merge all arg cues

check if the caller is a known sanitizer
/ source / pass-through

This will happen regardless as you
recursively go deeper.

↓
If yes, cleans away All taints only
THAT sanitizer is capable of clearing.
we attached a cue to each
sanitizer in **populate Maps()**, so it
knows exactly which cue to remove.

↓
Come back into **build Summary()**

↓
check if expr on RHS contains a
hardcoded source and/or it comes from
the parameter, bcz we already seeded the
param, the LHS will be tainted.

↓
if yes, add LHS Def to taint map.

↓
Hits return check (say, return x p

Runs **evaluate Expression Taint()** on x's Node & extracts
srcCues and paramDeps again

↓
if srcCues has anything in it,
it means this is producing move
taint.

OR

if paramDeps was anything, it
means the param taints are reaching
the final expr.

↓
Start the sink check

One
step

Similar to the flows explained above, look at the code and it will click.

InterProcedural Taint Analyzer

→ caller argument → callee param

PASS 1: Uses the code and call graphs to traverse the edges to find CALL edges

Extract callsite info, this gives us names of the caller and callee functions.

Extract the callee's summary at this stage, we only need the context (lineno) and the paramNames

loop of the argument positions in the caller func to see which callee param is meant for which arg

Init the calleeScope, and see which current argument cves are active.

Copy the arg cves onto the param variable's set of cve ids.

PASS 2: callee return val → caller variable

Loop over unconditionally tainted funcs, add whatever CWEs you find, as these funcs will ALWAYS return tainted stuff.



Loop over conditionally tainted funcs, add whatever we find.



Pass 3:

Same process as Pass 2 occurs for sinks



create vulnerability if found



return the vulnerability list so that developers have a paper trail.